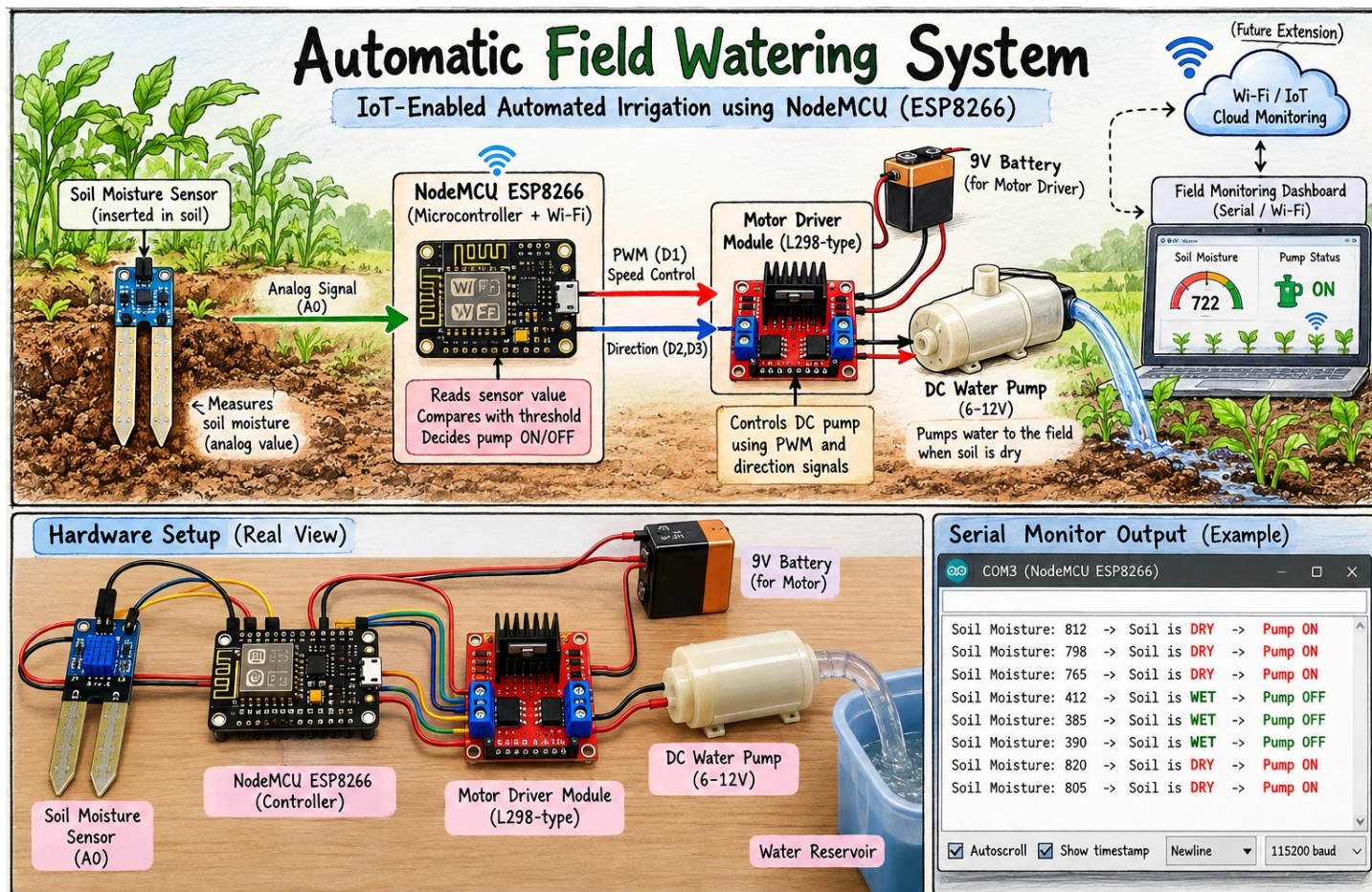


AUTOMATIC FIELD WATERING SYSTEM

IoT-Enabled Automated Irrigation using NodeMCU (ESP8266)



Prepared By: G Indranag

1. Abstract

Irrigation scheduled by habit rather than by the actual condition of the soil is one of the largest avoidable sources of water wastage in both small-scale farming and home gardening. A controller that senses soil moisture directly and waters only when the soil is genuinely dry removes the guesswork from this process and returns the decision to the plant's real-time need.

This project implements such a controller around a NodeMCU (ESP8266) development board. A resistive soil moisture probe is read through the board's analog input, and the resulting reading is compared against a fixed calibration threshold once every second. When the soil is found to be dry, the firmware drives an L298-type dual H-bridge motor driver module, which in turn switches on a small DC water pump at a fixed PWM speed; when the soil is found to be adequately wet, the same driver is commanded off. Every decision is also written to the serial monitor, which allows the threshold and the pump behaviour to be verified and tuned during development. The assembled system was tested on the bench using a metal reservoir of water in place of a field-scale supply. Dry-soil readings were observed to sit well above the calibration threshold, causing the pump to run continuously, while wetting the probe brought the reading below the threshold and stopped the pump on the very next sampling cycle. The project demonstrates a complete embedded control loop — analog acquisition, threshold-based decision-making, and motor actuation through PWM — and forms a directly extensible base for a networked, fully automatic irrigation controller.

2. Introduction

Every automated system, however small, is built on the same underlying cycle: sense the physical world, process the measurement, and act on the result. This project is a compact and complete realisation of that cycle for the specific problem of field and garden irrigation, chosen because each of its three stages — sensing, decision, and actuation — can be independently observed and verified on the bench before the system is ever taken to a real field.

Soil moisture is a convenient physical quantity to automate around for two reasons. First, it changes slowly — over minutes rather than seconds — so a modest one-second sampling interval is more than sufficient, and the system's behaviour can be watched comfortably in real time during testing. Second, it can be manipulated at will simply by moving the probe between dry air and wet soil or by adding water to the test reservoir, which makes the system easy to verify without any specialised laboratory equipment.

The ESP8266 was chosen as the controller in preference to a conventional AVR-based board for a specific reason: it carries an integrated 802.11 b/g/n Wi-Fi radio on the same die as the microcontroller. Although the present firmware does not yet use the radio, its presence means the system can be upgraded from a standalone, bench-tested irrigation controller into a networked, remotely monitored one purely through firmware changes, with no alteration whatsoever to the hardware already built.

3. Objectives

- Acquire a continuous, repeatable analog measurement of soil moisture using a resistive probe connected to the NodeMCU's ADC channel.
- Convert the raw analog reading into a binary dry/wet decision using a calibrated comparison threshold.
- Drive a DC water pump automatically, through a motor driver module, at a fixed and safe PWM speed whenever the soil is dry.
- Stop the pump automatically, with no manual intervention, as soon as the soil reaches an adequate moisture level.
- Stream the live sensor reading and the derived pump status over the serial interface for monitoring, debugging and threshold tuning.

- Verify correct operation experimentally under both dry and wet conditions and document the observed behaviour.
- Establish a hardware and firmware foundation that can later be extended to Wi-Fi telemetry or cloud-based remote monitoring without a hardware redesign.

4. System Block Diagram

The system is organised as a linear signal chain with a single decision point and a single actuated output. The soil probe converts the moisture content of the soil into an electrical property; the sensor module converts that property into a proportional analog voltage; the NodeMCU digitises the voltage, applies a decision rule, and commands the motor driver module, which in turn switches the DC pump on or off at a controlled speed.

4.1 Stage-by-Stage Description

Sensing stage: The resistive soil probe consists of two parallel conductive prongs inserted into the soil or reservoir. The soil or water between them completes a circuit whose conductivity depends on moisture content — wet soil conducts well, dry soil conducts poorly.

Signal conditioning stage: The sensor module places the probe in a potential-divider arrangement, converting the variable resistance into a variable analog voltage that swings across the usable input range of the NodeMCU's ADC.

Processing stage: The NodeMCU samples this voltage on its analog input pin, producing an integer reading, and compares it against the fixed threshold constant defined in the firmware to decide whether the soil is dry or wet.

Actuation stage: Based on this decision, the NodeMCU drives two digital direction pins and one PWM-capable enable pin on the motor driver module. The driver module, in turn, supplies the DC pump with current from the external 9 V battery, running it at the commanded speed or holding it off.

5. Hardware Requirements

Component	Specification	Purpose in this Project
NodeMCU ESP8266	ESP-12E module, 80 MHz Tensilica L106, 3.3 V logic, on-board USB-UART	Main controller: ADC sampling, threshold decision, PWM/direction output, serial reporting
Soil Moisture Sensor	Resistive probe with comparator breakout, analog output	Converts soil conductivity into a proportional analog voltage on pin A0
Motor Driver Module (L298-type)	Dual H-bridge, screw-terminal inputs/outputs, on-board heatsink	Drives the DC pump's speed (PWM on ENB) and direction (IN3/IN4) from 3.3 V logic signals
DC Water Pump / Motor	Small brushed DC motor, 6–12 V rated	Draws water from the reservoir and delivers it to the soil when energised
9 V Battery	PP3 alkaline / rechargeable pack	Supplies the motor driver's motor-side power rail, independent of the NodeMCU's logic supply
Water Reservoir	Small metal/steel container	Holds the water supply used to demonstrate pump operation on the bench
Jumper Wires	Male-to-male and male-to-female, assorted	Interconnect the NodeMCU, sensor module and motor driver
USB Cable	Micro-USB, data-capable	Powers and programs the NodeMCU, and carries the serial monitor link
Host Computer	Any PC with a free USB port	Firmware compilation, upload, and serial monitoring

5.1 Why These Component Choices Were Made

The motor driver module is used rather than driving the pump directly from a NodeMCU GPIO pin because a DC pump draws far more current, and briefly a much larger inrush current at start-up, than any microcontroller pin can safely source. The driver's H-bridge stage isolates the 3.3 V logic side from the higher-current motor side, while still allowing the logic pins to fully control the pump's on/off state and, through PWM on the enable pin, its effective speed. Powering the motor side from a separate 9 V battery rather than from the NodeMCU's own 5 V/3.3 V rails prevents the motor's switching noise and current demand from disturbing the microcontroller's supply.

6. Software Requirements

Item	Configuration Used
Development environment	Arduino IDE with the ESP8266 core installed through Boards Manager
Board selection	NodeMCU 1.0 (ESP-12E Module)
CPU frequency	80 MHz (default)
Flash size	4 MB (default FS/OTA partitioning)
Upload speed	115200 baud
Serial monitor baud rate	115200 baud
Additional libraries	None — the sketch uses only core Arduino functions (analogRead, digitalWrite, analogWrite)

As with any serial-based debugging setup, the serial monitor's configured baud rate must match the rate set in the firmware; a mismatch produces unreadable characters rather than the expected soil-value and status lines, and is one of the first things to check if the monitor output looks like garbage.

7. Pin Configuration and Circuit Connections

Signal	NodeMCU Label	Direction	Notes
Soil sensor analog output	A0	Input	Only analog channel available on the board
Motor driver ENB (speed)	D1	Output (PWM)	Sets pump speed via analogWrite, 0–255
Motor driver IN3 (direction)	D2	Output (digital)	Held LOW/HIGH together with IN4 to set direction / stop
Motor driver IN4 (direction)	D3	Output (digital)	Paired with IN3 for direction control
Motor driver power input	9 V battery +/-	Power	Independent supply for the pump, isolated from logic rail
Common ground	GND	Reference	Shared between NodeMCU, sensor module and motor driver

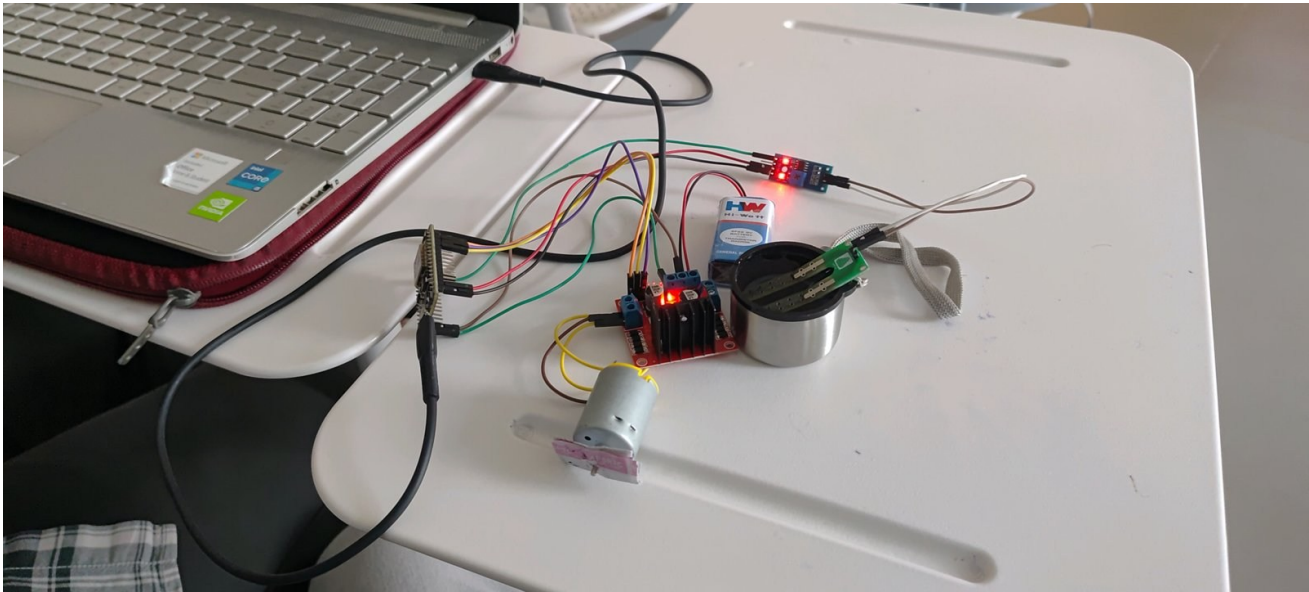


Figure 1 — Circuit connection diagram: NodeMCU, soil moisture sensor, motor driver, DC pump and battery wiring

7.1 Why D1, D2 and D3 Were Chosen

Not every GPIO on the ESP8266 is equally free to use. Several pins are tied to boot-mode strapping logic and can prevent the board from starting correctly if pulled to the wrong level at power-up. D1, D2 and D3 carry no such restriction in this design, support PWM output where required (D1/ENB), and have no alternate-function conflicts, which makes them a natural and reliable choice for the driver's enable and direction lines.

7.2 A Note on Driving the Pump Safely

The firmware always writes IN3 and IN4 to the same logic level (both LOW) whenever the pump should be stopped, rather than leaving either pin floating. This avoids an undefined H-bridge state and ensures the pump coasts to a clean stop rather than braking abruptly or drawing an unexpected current path.

8. Working Principle

The operating principle rests on the same physical fact used in most low-cost soil sensors: the electrical conductivity of soil rises with its water content, because water dissolves mineral salts into mobile ions that carry current between the probe's electrodes.

When the soil is dry, few mobile ions are present, the path between the probe's electrodes is highly resistive, and the sensor module's divider output sits near the upper end of its range, so the ADC reports a high reading. When the soil is wet, the same path becomes far less resistive, the divider output is pulled down, and the ADC reports a low reading. A higher analog reading therefore corresponds to drier soil.

8.1 Firmware Execution Sequence

On power-up, the firmware performs a one-time initialisation: it opens the serial port at 115200 baud, configures the enable and direction pins as outputs, and drives the pump off by holding both direction pins LOW and the PWM output at zero — so the system always starts from a known, de-energised state rather than from whatever level the pins happened to float to.

It then enters a continuous loop, repeating once per second, that performs three actions in order:

- Sample — the analog voltage on A0 is digitised into a soil-moisture reading and printed to the serial monitor.
- Decide — the reading is compared against the threshold constant.

- Actuate — if the reading exceeds the threshold the soil is dry: the direction pins are set to run the pump forward and the enable pin is driven with the fixed PWM speed, while the serial monitor prints 'SOIL DRY – MOTOR ON'. Otherwise, both direction pins are held LOW, the PWM output is set to zero, and the serial monitor prints 'SOIL WET – MOTOR OFF'.

A one-second delay closes the loop. This interval is deliberately chosen: it is far faster than soil moisture can meaningfully change, yet slow enough that the serial log remains easy to read and the pump does not switch rapidly on and off near the boundary.

9. Threshold Logic and Calibration

The threshold is the single most important tunable parameter in the system, since it alone determines the boundary between soil considered 'wet enough' and soil considered 'too dry to leave unwatered'.

ADC Range	Soil Condition	Pump State	Serial Output
Below threshold (≤ 550)	Wet / sufficient	OFF	SOIL WET – MOTOR OFF
Above threshold (> 550)	Dry / insufficient	ON (PWM speed 200)	SOIL DRY – MOTOR ON

9.1 Why This Threshold Value Was Used

The threshold of 550 was set empirically for the probe and soil sample used during development, sitting comfortably away from both the fully-dry and fully-wet extremes observed on the bench. Using a fixed, pre-characterised value rather than the raw ADC midpoint avoids false triggering caused by the specific probe's baseline offset and the sensor module's own component tolerances.

9.2 Field Calibration Procedure

For deployment with a different probe or soil type, the threshold should be re-determined empirically rather than assumed. The recommended procedure is:

- Insert the probe into thoroughly dry soil of the target type, run the sketch, and record the steady serial reading as the dry reference.
- Water the same soil to the moisture level considered ideal for the plant, allow it to settle for a few minutes, and record the new steady reading as the wet reference.
- Set the threshold to a value between the two references, closer to the dry reference so the pump favours turning on slightly early rather than late, then update the constant in the firmware and re-flash.
- Verify by cycling the probe between the two conditions several times and confirming that the pump switches cleanly each time.

9.3 Factors That Affect Calibration

Several variables shift the readings and mean a threshold calibrated for one situation may not transfer to another. Soil composition matters most — clay retains water and dissolved salts far better than sand, so identical water content produces different conductivity. Fertiliser content raises ionic concentration independently of water. Probe insertion depth and contact firmness also change the effective sensing area, and supply-voltage variation on the sensor module directly scales its output, so a board on a weak USB source may read differently from one on a solid supply.

10. Testing and Results

The assembled circuit was tested on the bench with the NodeMCU connected to a host PC over USB and the Arduino IDE serial monitor open at 115200 baud. The soil probe was cycled between a dry condition and the metal water reservoir, and the pump and serial output were observed at each state.

10.1 Test Case 1 — Dry Condition (Motor ON)

With the probe held in dry air, the analog reading remained above the 550 threshold across successive samples. The controller correctly energised the motor driver, the DC pump ran continuously at the fixed PWM speed, and the serial monitor printed 'SOIL DRY – MOTOR ON' on every one-second cycle.

10.2 Test Case 2 — Wet Condition (Motor OFF)

The probe was then placed in the water reservoir. The reading dropped promptly below the 550 threshold; the driver's enable output was set to zero and both direction pins were held LOW, stopping the pump immediately. The serial monitor changed to 'SOIL WET – MOTOR OFF' on the very next sampling cycle, confirming that the one-second loop delay is the only latency present in the system.

10.3 Consolidated Results

Test Condition	Threshold Comparison	Pump State	Serial Status
Dry soil / open air	Reading > 550	ON, continuous PWM drive	SOIL DRY – MOTOR ON
Probe in water reservoir	Reading ≤ 550	OFF, driver de-energised	SOIL WET – MOTOR OFF

10.4 Serial Monitor (Arduino IDE)

The screenshot shows the Arduino IDE interface with the 'plantino' sketch loaded. The Serial Monitor is open, displaying a series of log entries. Each entry consists of a soil value followed by a status message. The status alternates between 'SOIL DRY – MOTOR ON' and 'SOIL WET – MOTOR OFF' based on the soil value relative to a threshold of 550. The soil values are consistently above 550 for the 'DRY' state and below 550 for the 'WET' state.

```

21
22 void loop() {
23
24   int soilValue = analogRead(SOIL_PIN);
25
26   Serial.print("Soil Value: ");
27   Serial.print(soilValue);
28
29   if (soilValue > MOISTURE_THRESHOLD) {
30     // Soil dry -> Motor ON
31     digitalWrite(IN3, HIGH);
32     digitalWrite(IN4, LOW);
33     analogWrite(PWM_MOTOR_SPEED);
34   }
35 }

```

Output Serial Monitor x

Not connected. Select a board and a port to connect automatically.

Soil Value: 1024 | SOIL DRY – MOTOR ON
 Soil Value: 1024 | SOIL DRY – MOTOR ON
 Soil Value: 1024 | SOIL DRY – MOTOR ON
 Soil Value: 741 | SOIL DRY – MOTOR ON
 Soil Value: 486 | SOIL WET – MOTOR OFF
 Soil Value: 489 | SOIL WET – MOTOR OFF
 Soil Value: 493 | SOIL WET – MOTOR OFF
 Soil Value: 498 | SOIL WET – MOTOR OFF
 Soil Value: 501 | SOIL WET – MOTOR OFF
 Soil Value: 505 | SOIL WET – MOTOR OFF
 Soil Value: 494 | SOIL WET – MOTOR OFF
 Soil Value: 496 | SOIL WET – MOTOR OFF
 Soil Value: 500 | SOIL WET – MOTOR OFF
 Soil Value: 506 | SOIL WET – MOTOR OFF
 Soil Value: 490 | SOIL WET – MOTOR OFF
 Soil Value: 488 | SOIL WET – MOTOR OFF
 Soil Value: 497 | SOIL WET – MOTOR OFF
 Soil Value: 505 | SOIL WET – MOTOR OFF
 Soil Value: 509 | SOIL WET – MOTOR OFF
 Soil Value: 512 | SOIL WET – MOTOR OFF

Figure 2 — Serial Monitor Readings

10.5 Observations and Analysis

- State transitions occurred on the very next sampling cycle after the probe was moved, confirming there is no hidden latency beyond the deliberate one-second delay.
- The pump and the printed serial status always agreed with each other in every observed trial, confirming that the indication and the physical actuation are driven from the same decision and cannot diverge.
- No unintended intermediate state was observed — the direction pins and PWM output changed together as a single atomic update within each loop iteration.
- Running the pump against the sealed metal reservoir during testing produced no abnormal current draw or heating in the driver module over the short test duration used.

- Fully automatic operation — the pump switches on and off without any manual monitoring or intervention.
- Conserves water by irrigating only when the soil is genuinely dry, rather than on a fixed schedule.
- Low component count and cost, making the design well suited to classroom demonstration and small-scale field prototyping alike.
- Continuous serial logging exposes the raw soil reading, allowing the threshold to be recalibrated for any soil type without guesswork.
- Built on the ESP8266, so Wi-Fi connectivity is already present in hardware and can be enabled through firmware alone, with no hardware modification.
- The motor driver isolates the pump's current demand from the microcontroller, protecting the NodeMCU from electrical damage.
- probes corrode over time under continuous DC excitation, as electrolysis gradually erodes the electrode plating; capacitive probes are preferred for long-term field deployment.
- The single fixed threshold includes no hysteresis, so a reading sitting exactly at the boundary can cause the pump to toggle between states on successive samples.
- The ESP8266 provides only one ADC channel, so monitoring multiple zones or depths requires an external analog multiplexer or a separate ADC chip.
- The pump runs at one fixed PWM speed rather than a speed proportional to how dry the soil is.
- There is no data persistence — if the host PC or a future logging service is disconnected, the historical readings are lost.
- The system currently reacts only to the present reading and applies no weather or rainfall forecast information.

13. Future Enhancements

- Add software hysteresis using two thresholds — one for switching the pump on and a slightly lower one for switching it off — to eliminate boundary flicker entirely.
- Average several consecutive ADC samples before comparison to further suppress electrical noise and transient spikes.
- Publish readings and pump status over Wi-Fi using MQTT or HTTP to a cloud dashboard such as ThingSpeak or Blynk, enabling remote monitoring and historical charting.
- Add multiple soil probes across different field zones, each with its own threshold, for zone-specific irrigation.
- Integrate a real-time weather forecast API to suppress watering when rainfall is expected soon.
- Power the probe from a GPIO pin and energise it only during the moment of measurement, which dramatically reduces electrode corrosion and extends probe life.
- Migrate to a capacitive soil probe for improved long-term stability and immunity to electrode degradation.
- Add solar charging for the battery supply to allow fully off-grid, continuous field operation.

14. Conclusion

The automatic field watering system was successfully designed, assembled, programmed and tested. Under experimental verification, the firmware reliably distinguished between dry and wet conditions and correctly drove the DC pump on and off through the motor driver module in direct response to the soil moisture reading, with the pump and the serial telemetry agreeing with each other in every observed trial.

Beyond the immediate result, the project demonstrates a complete and correctly structured embedded control loop: analog acquisition through the ADC, threshold-based decision-making in firmware, and safe, isolated actuation of a DC motor through PWM and an H-bridge driver. Each of these stages is a building block that recurs in almost every embedded automation product, which makes the exercise valuable well beyond its specific application. The choice of the ESP8266 platform further means that the step from this standalone bench-tested controller to a networked, remotely monitored irrigation system requires only firmware development, with the existing hardware carried forward unchanged.

15. Project Video



AFWS Project Video.mp4

Note: Double Click on above file to open Video.